Metro Navigator POC — What Works and What Doesn't

Nov 3, 2026 · @kamlesh kumar Patel

The POC can give a usable position in a metro tunnel with no GPS and no network: in 508 of 510 simulated rides every station stop was identified, with the position exact at each stop and typically 12–36 m off between stations. It is not yet proven on a real metro, and without a saved line (on a road) the error grows to hundreds of metres within a few kilometres.

The short answer

On a metro line: yes, it can work in production, if four conditions hold. The line is saved on the phone (it downloads by itself when there is network), the phone sits fixed in its box, the train stops at stations, and there was one position (GPS or network) at or before the station where the ride starts. Every station stop then puts the position exactly on that station, so errors never build up for more than one run.

On a road, or anywhere without a saved line: only for short gaps. With no GPS, no network and no line, phone sensors alone drift by hundreds of metres within a few kilometres. That is physics, not a bug: nothing outside the phone corrects them.

Situation	Works?	What to expect
Metro line saved, box fixed, train stops at stations	Yes	Position exact at each stop; 12–16 m average error between stations; worst point of a ride usually under 200 m
Long stops (up to 30 s), air-conditioning keeps the phone "moving"	Yes	Stops found from the vibration drop plus the braking
Train stops at a signal in the tunnel	Yes	Not mistaken for a station
Express train skipping stations	Yes, less accurate	28–36 m average; worst point up to about 380 m before the next stop
Box knocked or moved at a platform	Yes	Ignored; this used to break all later stops

Situation	Works?	What to expect
Boarding underground (no GPS on the train)	Yes, after one fix at the entrance	Without any fix at all, the start station is unknown
Phone without an orientation sensor	Mostly	Stations right, speed error 3× larger (about 3 m/s)
Road, or no line saved, with no GPS and no network	Short gaps only	Simulation: 90 m average, 230 m worst over 3.6 km; real roads (traffic, turns) worse
Never had any position, no line	No	Nothing to start from
Better than about 50 m everywhere in a tunnel, all the time	No	Needs beacons or the train's own location feed

All numbers come from a physics simulator (390 rides) and road tests. **None of it has been tried on a real metro yet.** That is the one step that decides production.

Your requirement: the four options

The requirement is achievable with the third option, and that is what the POC is built on. The other three need something outside the phone.

Option in the requirement	Achievable?	In the POC
Estimation from coverage details (potentially trained)	Only with training (rides that learn each line's cells) or the operator's cell data	Not used for position: the training was removed at your request. The coverage is logged in every row (serving cell and its strength)
Additional beacons	Yes, with beacons installed in stations or tunnels and the operator's permission	Not built (Bluetooth removed at your request). The way to get accuracy between stations
All phone / Android built-in features	Yes	**Built:** GPS, network location (Wi-Fi and cell towers), accelerometer, gyroscope, compass, barometer, plus the saved metro lines

Option in the requirement	Achievable?	In the POC
Commercial solutions	Yes, with a contract: an indoor-positioning vendor, or the operator's train-location feed	Not something the app can do alone

Getting the position into Nemo Handy: built. With ⋮ › **Send position to Nemo Handy** on and logging running, the app gives its position to every app on the phone as the phone's own location once GPS has been gone for 10 s. It does this as a mock location on both the GPS provider and Google's fused provider, so it works whichever one Handy reads.

- **One-time setup on the phone:** Developer options › Select mock location app › Metro Navigator. Without it, the app shows a warning with a Fix button.
- **While GPS works**, nothing is replaced and Handy gets the real receiver.
- **Why it pauses:** while the GPS provider is replaced, Android switches the real receiver off for every app. So every 30 s the feed pauses for 8 s to look for real GPS, longer (up to 30 s) while satellites are being picked up. Back in the open, the real GPS takes over within about 45 s at most.
- **The log** marks each row whose position went to Handy ("Sent to Handy"), plus events when sending starts, pauses and stops.
- **Not during a tunnel test** (test drive), which needs the real GPS as its reference.
- **Tested on the emulator:** GPS silenced, then 10 s later both providers handed other apps this app's position (marked mock, accuracy = the app's error). After the 30 s check the real GPS came back and sending stopped. **Not yet tried with Nemo Handy itself.** Some apps ignore mock locations or read the receiver's raw NMEA sentences, so check Handy shows the position in a tunnel before relying on it.

Location order: GPS, then network, then dead reckoning

This order is now how the app works. On Android, "network" and "cell tower" location are one source: Google's fused location works out the position from the Wi-Fi access points and cell towers around the phone.

1. **Good GPS fix** (±50 m or better): this is the position, every second.
2. **Only poor GPS for 6 s**: GPS counts as lost. Poor fixes still give speed and direction (Doppler), which steer the estimate.
3. **Network location** (every 2 s): corrects the estimate, weighted by its accuracy (Wi-Fi about ±20–50 m, cell towers about ±300–1500 m). On a metro line it moves the position only along the line, and outliers are rejected.

4. **Dead reckoning**: on a line, the train's speed from the sensors, put back on the station at every stop; off a line, the gyroscope's heading.

Is fused location used properly? Yes, after two fixes. It runs at high accuracy every 2 s next to the GPS receiver; its fixes are ignored while GPS is good and take over when GPS is lost. Without Google Play services the phone's own network provider is used instead.

- Fixed: when only poor GPS fixes arrived, GPS still counted as working, so neither network nor sensors were used and the position froze.
- Fixed: fixes up to 30 s old were accepted, and a train covers 300 m or more in that time. The limit is now 10 s.

Cell towers with no data connection: not achievable underground. Google's network location needs mobile data for most lookups. Offline cell-tower positioning needs a database of tower positions (OpenCellID or a paid Google service). In a tunnel even that would mislead: tunnel antennas are fed from base stations registered somewhere else, often kilometres away. The only way around that was learning the tunnel's cells on earlier rides, the "training" removed in this round. The app still logs the serving cell and its strength in every row, so the coverage along the route is recorded.

What was wrong in the road test, and what was fixed

Seven faults explain the bad road-test track (missed stops, missed turns, wrong speed); three more turned up in the simulations. Each fix has a test that recreates the situation (`NoGpsDrivingTest`, `EdgeCaseTest`, `UndergroundScenarioTest`).

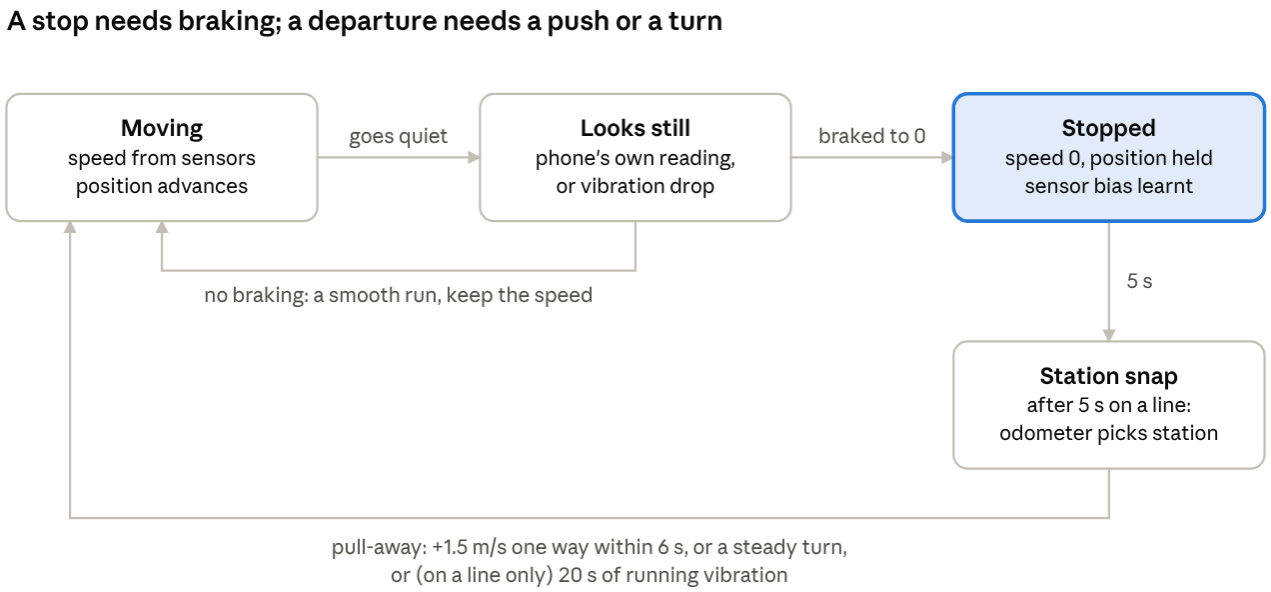
What you saw	Cause	Fix
Position froze under flyovers or tall buildings	Poor GPS fixes kept GPS "on" but were not used	GPS counts as lost after 6 s with no good fix; sensors and network take over
Position stuck after a stop where GPS was lost	Stopping with GPS made the app decide it was no longer in a vehicle, so pulling away was ignored	A vehicle that stops stays a vehicle; only a long walk at walking pace ends the ride
Position stuck after a gentle or turning pull-away	Only a strong, straight 4-second push counted as a departure	A departure is a steady push in one direction, or (on a road) a steady turn
Speed fell to walking pace on bumpy roads	Strong shaking was taken as walking (1.2 m/s, compass direction)	In a vehicle, shaking keeps the vehicle model; step counting removed

What you saw	Cause	Fix
Turns missed, heading drifting after stops	Slow turns that read as "still" could count as a stop after 15 s, and were learnt as gyroscope bias	A turning vehicle is never standing; turn rates over 1°/s are never taken as bias
Estimate could creep forward at a red light	Engine vibration read as "moving"	GPS lost while standing: only a real pull-away moves the estimate; off a line, vibration alone never does
Network location ignored after a stop	A standing estimate's error does not grow, so far-away network fixes were rejected for ever	Three network fixes in a row that agree with each other win
One missed stop hid every later stop (simulation)	Inertial drift made the braking check fail at each later stop	A braking check that drift cannot affect, and the sensor bias averaged over the whole ride
Same coordinates for up to 14 s near a station while still moving (simulation)	The estimate waited at the station when it arrived before the train	It slows down and keeps moving until the train stops
Coordinates moving during a stop when the box was knocked (simulation)	A knock could pass as a short push or a turn	A departure needs a push of 2 s or more, and a steady turn

The last fault was the worst: with the box knocked at every platform, 2 of 30 simulated rides ended up kilometres off (worst stop error 5.8 km). After the fix the worst stop error in the same 30 rides is 9 m.

How a stop and the next departure are found with no GPS and no network

A stop is accepted only when the phone looks still **and** the motion sensors show the train braked to a halt; a departure needs a real push or a turn. Vibration alone decides neither.



stop and departure · 4 states, no GPS or network needed

- **Looks still**: either the phone's own still reading, or the vibration falling below 40% of its level over the last half-minute of running (air-conditioning or an idling engine can keep the phone's reading on "moving").
- **Braked to 0**: the acceleration since the fastest moment of the last 25 s has removed nearly all of that speed. A smooth stretch at speed does not pass, nor does slowing down without stopping.
- **While stopped**: speed is 0, the position does not move, and the accelerometer and gyroscope biases are measured (whatever they read while standing is error).
- **Which station**: after 5 s still on a metro line, the inertial odometer (distance since the last station, within about 10%) picks the station and the position is put exactly on it. A stop far from any station counts as a signal stop and does not move the position.
- **Departure**: the train's push (+1.5 m/s in one direction within 6 s), a steady turn (a standing train does not turn), or, on a line only, 20 s of running vibration. Speed starts from the measured value, and the direction comes from the chosen destination, the last GPS course or the push itself.

In simulation the position stayed still for more than 90% of every dwell, including 30 s stops with air-conditioning, and every departure was picked up.

Same coordinates only during a real stop

Without GPS, the coordinates stay the same only while the train really stands; while it moves, every log row is a new position.

- **Near a station** the estimate used to wait at the station (up to 14 s of identical rows) when it arrived before the train. It now slows down and keeps moving until the train stops, then snaps to the station.
- **While standing**, network and poor-GPS fixes no longer nudge the coordinates. Only three fixes in a row that clearly disagree with the estimate correct it.
- **A knocked box** can no longer fake a departure: a departure needs a push one way lasting at least 2 s, and a turn must be steady.
- **Measured in 540 simulated rides:** the coordinates never moved while the train stood (0 s in every situation except right after a 60 s app restart, at most 0.9 s). While it moved, the longest run of identical coordinates was 1.5 s, while a pull-away was being confirmed (3.4 s on the first departure after boarding underground, while the direction is unknown).
- **On the road:** a whole 2.4 km test drive (cruising, braking, standing with the engine idling, pulling away, a right turn) gives a new position in every 2-second row while moving and the same position while stopped.

Built for an unattended box

A box nobody touches has to look after itself, so these situations are now handled.

Situation	What happens now	How it was tested
Train turns back at the end of the line	The next run is automatically the other way, whatever the compass says	60 simulated round trips: every stop right, even when the compass error changes after reversing
Wrong first direction (train starts where GPS cannot see it, compass far off)	The compass can set an unknown direction but never turns a known one round; network fixes and the line's curves correct a wrong one	Compass 180° off: with network, every stop right in 30 of 30 rides. Without network, it corrects itself at the first curve; the stops before that are wrong
App crashes while sending to Handy	The real GPS is put back at once, and Android restarts logging within about a second	Real crash on the emulator
App force-stopped while sending	Android keeps the phone's GPS replaced until the app is opened again, which cleans it up	Emulator. Force-stop needs a person, so it should not happen in a closed box

Situation	What happens now	How it was tested
App restarted in the middle of a ride	It carries on from the saved ride (line, position, direction, last station; saved every 5 s, used for up to 5 minutes), and the next stop puts it right	60 simulated restarts with 20 s and 60 s gaps: every stop right. On the emulator, nobody touched the phone: "Ride resumed" in the log, GPS back 4 s later
Box runs longer than 12 hours	The keep-awake is renewed every hour while logging	Lock checked on the emulator; the renewal itself not watched (it comes after an hour)

The automatic restart was tested on an Android 12L emulator. Newer Android versions are stricter about restarting location services in the background, so check it on the actual phone, with battery optimisation off for the app.

Edge cases measured

In 508 of 510 simulated metro rides every station stop was identified; in the other 2 (the same ride, with and without a gyroscope), one stop was taken for a signal stop and the position was 93 m off until the next stop. Each situation below is 30 rides on a 20 km curved line with 16 stops (32 for round trips), no GPS after the first 300 m and no network underground unless stated, with sensor noise, bias and a 25° compass error.

Situation	Every station right	Worst stop error	Average error between stations	Worst point (95% of rides below)	Speed error	Longest same coordinates while moving	Moved while standing
Normal ride, one speed	30 / 30	0 m	13 m	113 m	1.0 m/s	1.5 s	0 s
Different speed on every run	29 / 30	93 m	16 m	155 m	0.8 m/s	1.5 s	0 s
30 s stops, phone never reads still (air-con)	30 / 30	0 m	15 m	192 m	0.8 m/s	1.5 s	0 s
Express skipping 6 stations	30 / 30	0 m	28 m	205 m	1.2 m/s	1.5 s	0 s
Express, different speed on every run	30 / 30	0 m	36 m	380 m	1.1 m/s	1.5 s	0 s

Situation	Every station right	Worst stop error	Average error between stations	Worst point (95% of rides below)	Speed error	Longest same coordinates while moving	Moved while standing
40 s signal stop in the tunnel	30 / 30	0 m	14 m	112 m	0.9 m/s	1.5 s	0 s
Smooth running reads as still every 20 s	30 / 30	0 m	13 m	113 m	1.0 m/s	1.5 s	0 s
Box knocked at every platform	30 / 30	0 m	15 m	128 m	1.0 m/s	1.5 s	0 s
Compass off by 60°	30 / 30	0 m	12 m	110 m	0.9 m/s	1.5 s	0 s
No gyroscope	29 / 30	93 m	16 m	155 m	0.8 m/s	1.5 s	0 s
No orientation sensor	30 / 30	0 m	43 m	115 m	3.0 m/s	0 s	0 s
Boarded underground, one fix at the entrance	30 / 30	0 m	12 m	131 m	0.8 m/s	3.4 s	0 s
Round trip, turn-back at the end of the line	30 / 30	0 m	12 m	116 m	0.9 m/s	1.5 s	0 s
Round trip, compass error changes after reversing	30 / 30	0 m	17 m	113 m	1.6 m/s	1.5 s	0 s
App restarted mid-ride, 20 s gap	30 / 30	0 m	13 m	112 m	1.4 m/s	1.5 s	0 s
App restarted mid-ride, 60 s gap	30 / 30	0 m	17 m	166 m	0.9 m/s	1.5 s	0.9 s
Wrong direction at the start, network every 10 s	30 / 30	0 m	56 m	1762 m (before it is corrected)	3.1 m/s	3.4 s	0 s
No line saved: free dead reckoning over 3.6 km	—	—	90 m	229 m	1.5 m/s	1.5 s	0 s

"0 m" at stops means the position was put exactly on the station. The worst point of a ride is usually just before a stop, when the estimate has run ahead or fallen behind; the stop then corrects it.

Road situations, each an automated test that passes:

- Only poor GPS for 20 s while driving: GPS counted as lost after 6 s, position within 40 m.
- GPS and network lost during a 30 s stop, then a pull-away: moves again, within 25% of the distance driven.
- 90 s idling at a red light: the position does not move.
- Right turn out of a junction while speeding up: heading within 10°.
- 20 s slow turn that reads as still: not taken for a stop, heading within 15°.
- Bumpy road that reads as walking: vehicle speed kept.
- Box knocked at a stop: no false departure.
- Network location disagrees three times with a standing estimate: the estimate moves to it.
- Network location while GPS is good: ignored.

All 98 automated tests pass, including a whole 2.4 km road drive and six metro situations that check the coordinate rule above. The simulator's sensors are a model: real phones, mounts and trains will differ, which is why the real-ride step below matters.

What has been achieved

A phone in a box gives a position underground on its own: no installation in the tunnel, no server, and nothing learnt or stored between rides.

- **Source order** GPS → network (Wi-Fi and cell towers) → dead reckoning, visible in every log row (`GPS`, `NET`, `TRACK`, `DR`).
- **Station stops**: found with no GPS and no network, matched to the right station by the inertial odometer, and used to reset the position; signal stops and express trains running through stations are handled.
- **Speed** from the motion sensors (acceleration plus a cruise model, cross-checked against the integrated velocity), zero while standing.
- **Turns**: on a line, the line's curves (and a gyroscope check against them); off a line, the gyroscope turns the heading with the vehicle.
- **Metro lines** of any city downloaded automatically from OpenStreetMap when there is network, plus 18 Indian cities built into the app, so no network is needed underground.
- **Log and map**: a new file each time logging starts, a row only for a new position (plus one every 30 s while standing), events, sensor state and the serving mobile cell; coloured markers on the map. Without GPS, the position can also go to Nemo Handy as the phone's own location (menu switch).

- **Test drive** without a metro: record a road route with GPS, ride it again with GPS hidden, and measure the error.
- **Removed** at your request: the self-learning (Wi-Fi and cell fingerprints, learnt run speeds and stillness), Wi-Fi scanning and ranging, step counting, and earlier the Bluetooth beacons. The app no longer asks for the Physical activity, Wi-Fi or Nearby devices permissions.
- **Checked by** 98 automated tests and 540 simulated rides across 18 situations. The app's code is 11 files.

What cannot be achieved, and what it would take

The phone alone cannot know where it is between two stops; it can only carry the last known position forward, and that error grows until the next stop.

Not achievable with the phone alone	Why	What it would take
An accurate position at every moment in a tunnel (better than about 50 m between stations)	Sensors drift; nothing outside the phone corrects them between stops	Beacons in the tunnel or at platforms (requirement R4), or the train's own location feed
Starting with nothing known (no fix ever, no line)	The start station must be known	One GPS or network fix at the entrance, or the user picks the station
Long trips off a metro line with no GPS and no network	No line to hold the position, no stops to reset it	GPS or network; road map-matching would be a large separate project
Cell-tower positioning with no data connection	Needs a tower database, and tunnel antennas are registered at distant base stations	Learning the tunnel's cells on earlier rides (the removed training), or operator data
Every stop right on express trains with varying speeds	The odometer is about 10% accurate; in about 1 ride in 30 one stop is misread	Tuning on real rides; beacons at stations
The same results on every phone model	Sensor quality and mounts vary	Testing on the phones and box actually used

Requirement NH-3779 R4 ("using additional beacons") and the "commercial solutions" option are not implemented: Bluetooth was removed at your request.

Before production

The deciding step is a real metro ride; everything else here is preparation or tuning.

1. **Ride a real metro line** end to end two or three times with the box and logging on. Compare each "Stopped at" event in the log with the real station, and on a line with an elevated stretch, compare with GPS there.
2. **Use the target phone and box**: check that stops at platforms are found, including with air-conditioning running, and that Nemo Handy shows the position in the tunnel with the feed on.
3. **Check the line data** for the city (OpenStreetMap): station positions and order.
4. **Decide the accuracy target.** If the position between stations must be better than about 100–200 m, plan beacons or the train's own location feed now.
5. **Run 8 hours** with the screen off and battery optimisation off for this app, to confirm logging never pauses.
6. **Tune from the real logs**: the vibration drop (40%), the braking tolerance and the odometer tolerance (10%).
7. **Commit and version the code.** Nothing from this round is committed yet.